Project Work Breakdown: Custom 2D Platformer

1. Project Overview

- **Objective:** Develop a custom 2D platformer game using C++ with a strong emphasis on Object-Oriented Programming (OOP) principles.
- **Tech Stack:** C++, SFML 2.x for rendering and window management.
- **Core Requirements:**
    - Strict application of OOP concepts (inheritance, polymorphism, encapsulation, abstraction).
    - Implementation of at least 5 design patterns (e.g., Factory, Singleton, Observer) to ensure a modular architecture.
    - Completion of 3 distinct levels with increasing difficulty.
- **Team Members:** Minh Khoa, Khải, Long, Đăng Khoa.
- **Duration:** 9 Weeks. (Note: Weeks 4 and 7 are designated as light-duty weeks, limited to 4-5 hours per member).

2. Task Division (Work Breakdown Structure)

This division ensures a balanced workload, allowing each member to handle core mechanics while securing the required design patterns.

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
Minh Khoa	**Core Engine & Physics**	Singleton, State Pattern	• Initialize SFML RenderWindow and event polling system. • Implement a fixed-timestep Game Loop for consistent physics. • Design the base State class and implement specific states (Menu, Playing,

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
			Paused) using the State Pattern. ● Code AABB (Axis-Aligned Bounding Box) collision detection algorithms. ● Apply gravity, velocity, and resolve entity-environment collisions (preventing wall clipping).
Khải	**Hero & Items Mechanics**	Factory, Strategy Pattern	● Define the abstract base Character class (health, speed, bounding box). ● Implement the Hero class with SFML sprite rendering and animation frames. ● Map keyboard inputs to movement, jumping, and action logic. ● Define the Item interface and concrete classes (e.g., Health Potion, Energy Shield). ● Implement the

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
			Factory Pattern for dynamic item spawning. ● Handle Hero state changes (Strategy Pattern) upon item collection.
Long	**Enemies & AI**	Factory, Template Method	● Inherit the Character class to create the base Enemy abstract class. ● Implement specific enemy types (e.g., melee chaser, ranged attacker). ● Utilize the Factory Pattern to generate various enemy types dynamically. ● Implement grid-based pathfinding algorithms (e.g., BFS or Dijkstra) for AI tracking. ● Code basic patrol routines using the Template Method. ● Manage enemy health, damage dealing, and

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
			death animations.
Đăng Khoa	**Level, UI/UX & Audio**	Observer, Singleton Pattern	• Create a text or CSV file parser to load map grid data. • Render the map using SFML TileMap (Vertex Arrays) for optimized performance. • Design the UI/HUD overlay (score, lives, health bar). • Implement the Observer Pattern to listen for score/health changes and update the UI independently. • Build a SoundManager class using the Singleton Pattern for BGM and SFX. • Implement serialization for Game Save/Load functionality via file I/O.

3. Project Timeline

Note: Regular weeks require 8-10 hours per member. Weeks 4 and 7 are limited to 4-5 hours per member.

Week 1: Foundations & Concept (8-10 hrs)

- **All Members:**
    - Research and learn game coding fundamentals (game loops, rendering cycles).
    - Study Core OOP principles (Inheritance, Polymorphism, Encapsulation, Abstraction).
    - Research and select the 5 specific Design Patterns to be applied.
    - Brainstorm and finalize the custom game theme, art style, and core gameplay mechanics.

Week 2: Core Skeleton & Initialization (8-10 hrs)

- **Minh Khoa:** Initialize the C++ project, configure SFML/CMake environments. Implement the primary game loop and basic gravity/collision mechanics.
- **Đăng Khoa:** Source or design initial graphical assets (sprites, tilesets). Develop the tilemap renderer to display a static level from a data file.
- **Khải:** Code the base Character class and the Hero class. Bind basic keyboard inputs for movement and jumping.
- **Long:** Structure the base Enemy class and prepare the AI movement logic framework.

Week 3: Entities & Interactions (8-10 hrs)

- **Khải:** Utilize the Factory pattern to spawn items (e.g., health, shields) and code collection logic.
- **Long:** Spawn custom enemies using the Factory pattern and implement basic patrol logic.
- **Minh Khoa:** Integrate complex collision detection between the Hero, Enemies, and Items.

Week 4: Refactoring & UI Integration (4-5 hrs - Light Week)

- **All Members:** Conduct code review, ensure strict OOP adherence, and clean up the architecture.
- **Đăng Khoa:** Implement the Observer pattern to update the UI (score, health) dynamically without cluttering the main game loop.

Week 5: Advanced AI & Mechanics (8-10 hrs)

- **Long:** Upgrade enemy AI (e.g., utilizing pathfinding algorithms like Dijkstra to chase the player or ranged attack logic).
- **Khải:** Finalize state changes for the Hero (e.g., invincibility frames, power-up visual

changes).
- **Minh Khoa:** Refine hitboxes to accurately match the custom sprite dimensions.

Week 6: Level Design & Expansion (8-10 hrs)

- **Đăng Khoa & All Members:** Fully construct and balance the 3 required levels with increasing difficulty. Place enemies and items strategically.
- **Khải:** Implement a character selection screen if multiple hero types are available.

Week 7: Playtesting & Audio (4-5 hrs - Light Week)

- **All Members:** Intensive playtesting across all 3 levels. Log bugs, fix clipping issues, and refine the gameplay feel.
- **Đăng Khoa:** Integrate background music and sound effects (SFX) using a Singleton Sound Manager.

Week 8: Polish & Documentation (8-10 hrs)

- **Đăng Khoa:** Finalize the Save/Load functionality using file handling.
- **Minh Khoa & Khải:** Perform memory leak checks and finalize code modularity/readability.
- **All Members:** Compile the Design Documentation, detailing class diagrams and design pattern explanations.

Week 9: Buffer & Delivery

- **All Members:** Final bug squashing and final code review.
- **Minh Khoa & Đăng Khoa:** Record and edit the final gameplay demo video.

4. Workflow & Source Control

(To be defined: Branching strategy, commit conventions, and pull request review process on GitHub).
Github URL: [trannhukhai98438/CS202-Group03-2DGame]
Branching Strategy:
The repository is structured around two main branches to separate the source code from the project documentation:

- `dev` ***branch:*** *The primary branch for all development and coding. Every team member must create their own feature branches (e.g.,* `feature/hero-movement`*,* `feature/enemy-ai`*) branching off from* `dev`*. Once a task is completed, it should be merged back into* `dev` *via a Pull Request.*
- `docs` ***branch:*** *Dedicated exclusively to project documents. This includes the design documentation, weekly reports, work division structures, and any other non-code files.*

Commit Conventions:

To maintain a clean, readable, and trackable version history, all team members must adhere to the following standard commit prefixes:

- feat: *for adding a new feature or functionality (e.g.,* feat: implement Factory pattern for items*).*
- fix: *for fixing a bug (e.g.,* fix: resolve character clipping through tilemap*).*
- docs: *for documentation updates (e.g.,* docs: add week 3 progress report*).*
- refactor: *for code changes that neither fix a bug nor add a feature, but improve code structure (e.g.,* refactor: reorganize Character class inheritance*).*
- style: *for formatting changes, removing white spaces, or adding comments that do not affect the code's logic.*

5. CMake Instructions

To compile and build the SFML C++ project, ensure you have CMake (version 3.10 or higher) and a compatible C++ compiler installed. Run the following commands in your terminal from the root directory of the project:

Step 1: Create and enter the build directory
This ensures that all compiled files are kept separate from the source code.
- mkdir build
- cd build

Step 2: Generate the build system files
This command tells CMake to look for the CMakeLists.txt file in the parent directory and configure the project.
- cmake ..

Step 3: Compile the project
This builds the actual executable based on your system's default build tool (Make, MSBuild, Ninja, etc.).
- cmake --build .

Step 4: Run the game
Once the build is successful, execute the compiled file. (Note: The exact executable name depends on how it is defined in your CMakeLists.txt).
- On Linux / macOS: ./CustomPlatformer
- On Windows (PowerShell/CMD): .\Debug\CustomPlatformer.exe